1. Method vs Constructor

Feature	Method	Constructor
Purpose	Performs specific actions/operations	Initializes an object when created
Return Type	Must have a return type (or void)	No return type, not even void
Name	Any meaningful name	Must have the exact same name as the class
Calling	Explicitly called using object	Automatically called when object is created (`new` keyword)
Inheritance	Can be inherited by subclasses	Cannot be inherited
Example	`void display() { }`	`Student() { }`

2. Pass by Value vs Pass by Reference

Feature	Pass by Value	Pass by Reference
Definition	A copy of the variable's value is passed	The actual memory address (reference) is passed
Effect on Original	Changes inside method do NOT affect original variable	Changes inside method DO affect original variable
Java Support	Java uses only pass by value for all types	Not directly supported in Java
Primitive Types	Value is passed directly	Not applicable

Feature	Pass by Value	Pass by Reference
Objects	Reference value is passed (copy of reference), not the object itself	Actual object reference is passed
Example	`void change(int x){ x=10; }` - original unchanged	`void change(Student s){ s.name="John"; }` - original changes

3. Public vs Private vs Protected

Feature	public	private	protected
Access Level	Most accessible	Least accessible	Moderate accessibility
Same Class	✓ Yes	✓ Yes	✓ Yes
Same Package	✓ Yes	✗ No	✓ Yes
Subclass (Different Package)	✓ Yes	✗ No	✓ Yes
Non-subclass (Different Package)	✓ Yes	✗ No	✗ No
Best Used For	Interface methods, constants	Internal implementation details	Inheritance, framework hooks
Example	`public void show()`	`private int balance`	`protected void calculate()`

4. Instance Variable vs Static Variable

Feature	Instance Variable	Static Variable
Keyword	No static keyword	Uses `static` keyword
Memory Allocation	Separate copy for EACH object	One copy shared by ALL objects
Access	Only via object reference (`obj.variable`)	Via class name (`ClassName.variable`) or object
Lifetime	Created when object is created, destroyed when object is destroyed	Created when class loads, destroyed when program ends
Value Change	Changing in one object doesn't affect other objects	Changing affects all objects
Use Case	Object-specific data (e.g., `name`, `age`)	Common data for all objects (e.g., `schoolName`, `count`)
Example	`String name;`	`static String college = "ABC";`